

Bits $\{x, y \in \mathbb{I}_{+}^{\infty}; x \geq y\}$ (~~AAA~~)

$$\text{NOT}(x) = \sum_{n=0}^{\lfloor \log_2 x \rfloor} 2^n \left(\left(\left\lfloor \frac{x}{2^n} \right\rfloor \bmod 2 + 1 \right) \bmod 2 \right) = \left(2^{\lfloor \log_2 x \rfloor + 1} - 1 \right) - x$$

$$x \text{ AND } y = \sum_{n=0}^{\lfloor \log_2 x \rfloor} 2^n \left(\left\lfloor \frac{x}{2^n} \right\rfloor \bmod 2 \right) \left(\left\lfloor \frac{y}{2^n} \right\rfloor \bmod 2 \right)$$

$$x \text{ OR } y = \sum_{n=0}^{\lfloor \log_2 x \rfloor} 2^n \left(\left\lfloor \frac{x}{2^n} \right\rfloor \bmod 2 + \left\lfloor \frac{y}{2^n} \right\rfloor \bmod 2 - \left(\left\lfloor \frac{x}{2^n} \right\rfloor \bmod 2 \right) \left(\left\lfloor \frac{y}{2^n} \right\rfloor \bmod 2 \right) \right)$$

$$x \text{ XOR } y = \sum_{n=0}^{\lfloor \log_2 x \rfloor} 2^n \left(\left(\left\lfloor \frac{x}{2^n} \right\rfloor + \left\lfloor \frac{y}{2^n} \right\rfloor \right) \bmod 2 \right)$$

Boolean Algebra

- $x \& y = y \& x$ (AND)
- $x \& (y \& z) = (x \& y) \& z$
- $x \& 0xFFFF = x$
- $x \& 0 = 0$
- $x \& x = x$

- $x | y = y | x$ (OR)
- $x | (y | z) = (x | y) | z$
- $x | 0 = x$
- $x | 0xFFFF = 0xFFFF$
- $x | x = x$

(NOT)

- $\sim(\sim x) = x$

XOR

- $x \wedge y = y \wedge x$
- ~~$x \wedge (y \wedge z) = (x \wedge y) \wedge z$~~
- $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$
- $x \vee 0 = x$
- $x \vee x = x$
- $x \vee \sim x = 1$
- $x \vee 0xFFFF = \sim x$

$$x \wedge x = 0$$

$$x \vee 0xFFFF = \sim x$$

$$a \wedge b = (a | b) \& (\sim a | \sim b)$$

$$a \vee b = (a \& \sim b) | (\sim a \& b)$$

Invertible

NOT, XOR, rl, rr

Non-Invertible

AND, OR, sl, sr.

$$x | (x \& y) = x$$

$$x \& (\sim x | y) = x$$

$$\sim(x | y) = \sim x \& \sim y$$

$$\sim(x \& y) = \sim x | \sim y$$

$$x | (y \& z) = (x | z) \& (x | y)$$

$$x \& (y | z) = (x \& y) | (x \& z)$$

$$x \& (y \vee z) = (x \& y) \vee (x \& z)$$

$$x \vee y = (\sim x \& y) | (x \& \sim y) | (x \& y)$$

$$x - y = \sim(x \vee y)$$

$$\phi^1((\phi y y) > S)$$

$$B \geq 0, M \geq 0, S \geq B$$

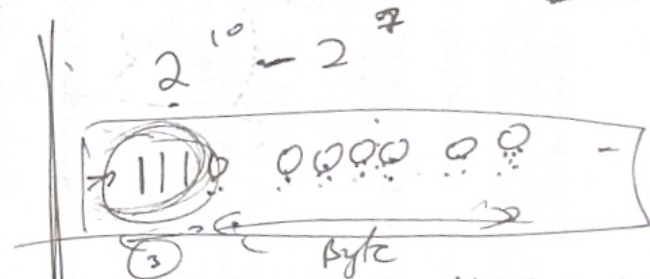
(3+4+2) 10

$$\Rightarrow \phi^1 \left(\left\lfloor \frac{(\phi y y)}{2^S} \right\rfloor \right) \Rightarrow \phi^1 \left[\frac{\phi \phi (2^{M+S} \cdot (2^B - 1))}{2^S} \right]$$

$$\Rightarrow \phi^1 \left[\frac{2^{M+S} (2^B - 1)}{2^S} \right] \Rightarrow \phi^1 \left[\frac{\phi \phi (2^{M+B+S} - 2^{M+S})}{2^S} \right]$$

$$\phi^1((\phi \phi (bm \ll (M+S))) > S)$$

$$\Rightarrow \phi^1((\phi \phi (\{ (1 \ll B) - 1 \} \ll (M+S))) > S)$$



$$(128 - 143) \leftrightarrow (144 \leftrightarrow 159)$$

$$(176 - 191) \leftrightarrow (160 - 175)$$

$$ABC: \phi \phi \phi \phi \phi \phi \phi \phi \text{ Mark}$$

$$\Rightarrow \underline{ABC}: 0000 000$$

$$2^7 + 2^9 + 12^9$$

$$128 + 256 + 512$$

$$640 + 256$$

$$896$$

$$128 \times 7$$

$$80 + 40 + 7$$

$$127$$

$$(128, 256, 512)$$

$2^3 = 8$

$(812, 256, 128) \vdots$

$(128, 64, 32) \vdots (64, 32, 16)$

000	Val	765	0	654	0
000	0	000	0	000	0
001	128	001	32	001	16
010	256	010	64	010	32
011	384	011	96	011	48
100	512	100	128	100	64
101	640	101	160	101	80
110	768	110	192	110	96
111	896	111	244	111	112

(0101010101010110)

$ab \bmod n b \equiv (a \bmod n) b$

$\{000ABC0000\}^1 \{ABC: \text{XXXXXX}\}$

$\downarrow \downarrow \downarrow$
 P O R

$ABC \vdots \{ABC^1POR\} \vdots \dots$
 $\downarrow \downarrow \downarrow$
 $812 \quad 256 \quad 128$
 $\downarrow \downarrow \downarrow$
 $64 \quad 32 \quad 16$

Things to know

- 32-bit: legacy addressing, limits to 4 GiB.
- 64-bit: Modern computers (laptops are 64 bit but 48 to 57 bits actually used for virtual translation, remaining bits for other stuff).
- 40-52 bit is the standard wiring in CPU/GPU hardware (2020¹⁸).
- 128-bit: Experimental "fat pointers" (CHERI) are under consideration and development for secure stuff.

* with only "+" | $c = a \times b$

```
c = 0
while (b != 0) {
    if (b < 1) != 0 {
        c = c + a
        a << 1;
        b >> 1;
    }
}
```

ret c;

* +1

```
while(a != 0) {
    c = b & a
    b = b ^ a
    c << 1;
    a = c; ret b;
}
```

Interleaved $\Rightarrow$ ab ab ...
 (4GB of data) $\Rightarrow$ 'abcdefghg.....', 'abc...g',
 1. Non-Interleaved memory (Linear / contiguous?)

- Bank A: Addr 0 to 1GB.
- Bank B: Addr 1GB to 2GB.
- Bank D: Addr 3GB to 4GB.

1. Interleaved

- Bank A: 128 Byte (0 - 128 byte - 1)
- Bank B: next 128 Byte (128 Byte - ...)

Tensor dimension and format

- Bit sized: .b32, .b64
- Sub-type types: .b4x16, .b4x16-64, .b6x16-p32, .b6p2x16
- Integers: .u8, .u16, .u32, .s32, .u64, .s64

Floating: .f16, .bf16, .tf32, .f32, .f64 (rounded to nearest even)

TENSORS

properties:

- Dimensionality
- Dimension across each dimension
- Individual element types
- Tensor stride across each dimension.

$$T = \{D_0 S\}$$

$$p_0 \{D_0 S\}$$

$$p^x \{D_0 S\}$$

$p1x$ supports the following operations:

- $\rightarrow$ Copy data b/w mem $\leftrightarrow$ mem
- $\rightarrow$ Reducing dst. data w/ src

Tensor may have padding in each of the dim. to ensure alignment in the subsequent dim. Tensor stride may be used to specify padding

bank offset $\Rightarrow \left(\left\lfloor \frac{\text{addr}}{2^7} \right\rfloor \right) \otimes 0x7 \rightarrow \text{hex}$

$\Rightarrow \left(\left\lfloor \frac{\text{addr}}{128} \right\rfloor \right) \otimes 0x1 \rightarrow (127)$

~~AA~~
secret

$(\text{addr} / 128) \% 8$ ~~AA~~

interleaved memory Banks: If the system has 8 physical memory banks model, works in parallel, it might route chunk 0 to Bank 0, chunk 1 to Bank 1, and so on...

($2^8 = 8 = 1024$ bytes)

or

Sub-rotation $\rightarrow$ Permutation for rotated numbers...

Bits

$V_8 = \text{Byte} = 2^8 \text{ combinations; } 2^8 \text{ values}$

256

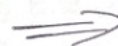
$2^{32} = 2147483648$

(int max)

$i \rightarrow i \bmod 8$
 $\lfloor \frac{i}{8} \rfloor$

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39
40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55
56	57	58	59	60	61	62	63

Swizzle



0	1	2	3	4	5	6	7
9	8	11	10	13	12	15	14
18	19	16	17	22	23	20	21
27	26	25	24	31	30	29	28
36	37	38	39	32	33	34	35
45	44	47	46	41	40	43	42
54	53	52	53	50	51	48	49
63	62	61	60	59	58	57	56

Grid-in → Punched holes marked numbers 0-9

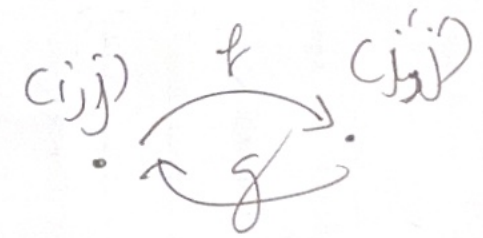
⊕	0	1	2	3	4	5	6	7
0	0 000	1 001	2 010	3 011	4 100	5 101	6 110	7 111
1	1 001	0 000	3 011	2 010	5 101	4 100	7 111	6 110
2	2 010	3 011	0 000	1 001	6 110	7 111	4 100	5 101
3	3 011	2 010	1 001	0 000	7 111	6 110	5 101	4 100
4	4 100	5 101	6 110	7 111	0 000	1 001	3 011	2 010
5	5 101	4 100	7 111	6 110	1 001	0 000	2 010	3 011
6	6 110	7 111	4 100	5 101	2 010	3 011	0 000	1 001
7	7 111	6 110	5 101	4 100	3 011	2 010	1 001	0 000

then
(xor d)

⊕ (i, j)

↓ f
(i', j')

↓ g
⊕ (i, j)



→ symmetric matrix

128 bytes = 1024 bits
 $2^7 \times 2^3$

Cute Swizzle

B, M, S

$$\text{bit-mask} = (1 \ll B) - 1$$

$$\text{yyy-mask} = \text{bit-mask} \ll (M + \max(0, S))$$

$$\text{zzz-mask} = \text{bit-mask} \ll (M + \min(0, S))$$

$$\text{maskshift} = S$$

apply &

return: offset ^ shifter (offset a yyy-mask & maskshift { })
 $\phi \wedge (\phi \& y) \gg S$

3

3, 4, 3

$i \wedge N + j = ?$

$(i \wedge N + j) \wedge ((i \wedge N + j) \& \text{yyy-mask}) \gg S$

$$\text{bit-mask} \rightarrow 2^{1 \ll B - 1} = 2^3 - 1 = \textcircled{7} = 0x0111$$

$$\text{yyy-mask} \rightarrow$$

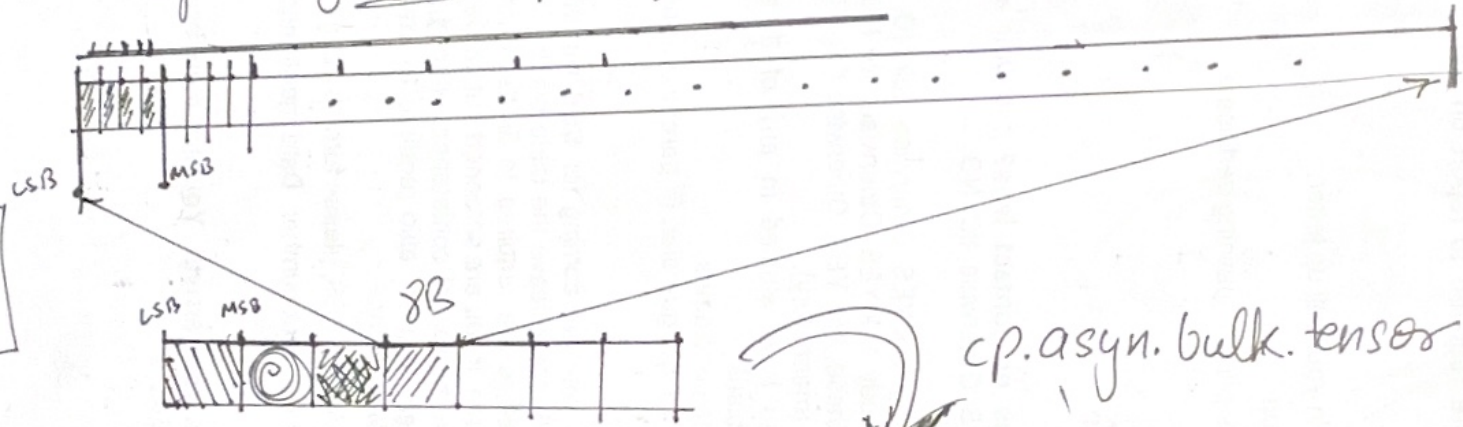
$$\text{zzz-mask} \rightarrow$$

Sub-bytes $\rightarrow$ Expected to be packed continuously in gmem, the tensor copy instruction ~~may~~ will expand by adding empty space $\otimes$.

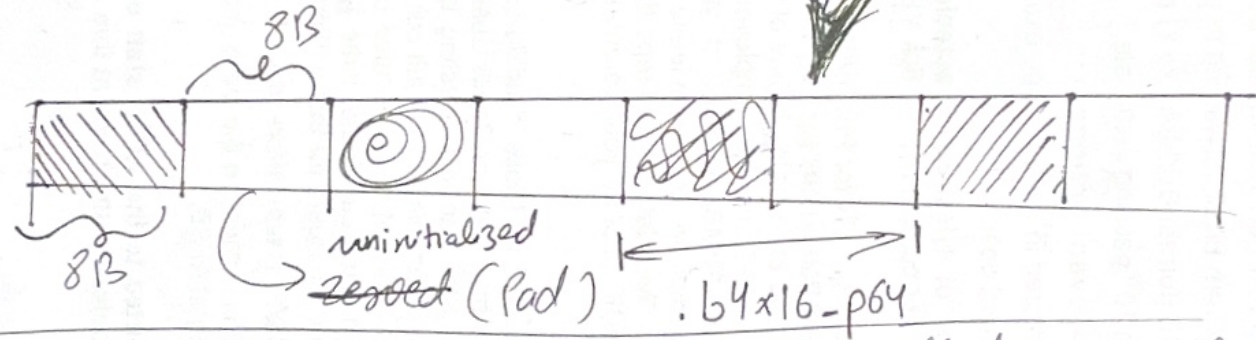
$$1.2 \times 16 = 18.4$$

gmem $.64 \times 16$

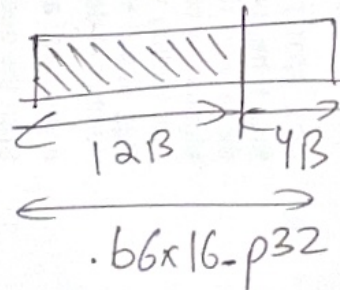
Has no padding in gmem as smem as it fits!



smem



for ~~the~~ $b6 \times 16 - p32$, we'll have 6 bits, the block will be 12 byte, hence the pad vs data segmentation will look as follows:



$\langle 0, \underline{4}, 3, 7 \rangle$
 $\Rightarrow \phi^1$

$$\left[\frac{\phi a(2^7 - 2^7)}{2^5} \right]$$

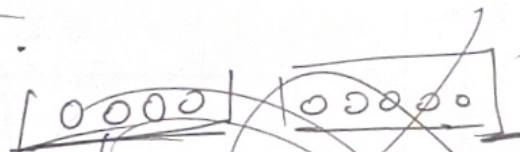
$\Rightarrow \phi^1 0 = \phi$

$\langle 1, 4, 3 \rangle$
 p m s

$$\phi^1 \left[\frac{\phi a(2^8 - 2^7)}{2^5} \right]$$

$2^4 \rightarrow 16 = 2 \times 8 \text{ bytes!}$

atomicity



$\rightarrow 2^4 = 16$

16

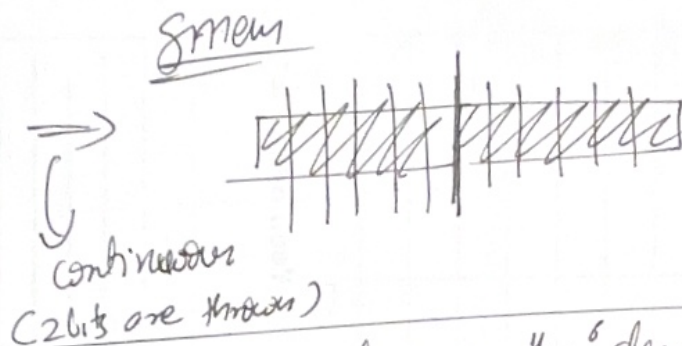
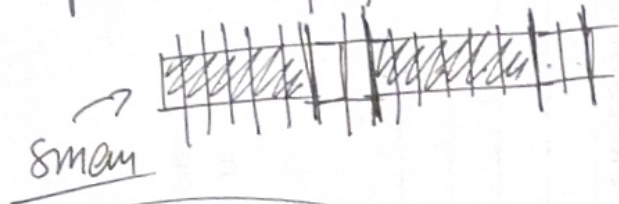
128

(+1) $\rightarrow$ 1 byte worth

~~256~~ -

$256 - 128 = 128$

Ex. $.b6p2 \times 16$ ^{2bits (pad)}



$.b6 \times 16 - p32$, $.b6p2 \times 16$: Both share same value in the 'descriptors'

Valid copy direction : $.shared :: cluster.global$, $.shared :: cta.global$.
 $.global.shared :: cta$ | Bounding Box : Chunk in which tensor is accessed.

Tensor have two access mode :

⊛ Tiled Mode : As is layout

⊛ Strided mode : Elements of Bounding Box gets rearranged into cols at dst.

- ⊛ Size of BB has to $n \cdot 16 \text{ bytes}$,
- ⊛ addr has also be 16 byte aligned,

Bounding Box Property :- (Dimensionality (same as accessing tensor))

- (Dimension sizes)
- (COB access mode)
- Transposed strides

$${}^n P_r = \frac{n!}{(n-r)!}$$

$${}^4 P_3 = \frac{4!}{(4-3)!} = 4 \times 3 \times 2 \times 1 = 24$$

$$4 \times 3 \times 2 \times 1 = 24$$

Swizzle from Layout (1286)

mode	ld / Mjones	
128B	M/N	8x8
	K	8x8
64B	M/N	4x8
	K	8x4
32B	M/N	2x8
	K	8x2
None	M/N	1x8
	K	8x1

$$8 \times \left(\frac{128}{32} \right) \times 8 = 32 \times 8$$

TMA/Cutlass name

No Swizzle

SW32

SW64

SW128

Cute byte address swizzle (B, M, S)

$\langle 0, 4, 3 \rangle$

$\langle 1, 4, 3 \rangle$

$\langle 2, 4, 3 \rangle$

$\langle 3, 4, 3 \rangle$

16-byte chunks.

1

32B

64B

128B

$$a' = a \oplus ([a \gg (M+S) \bmod (2^B-1)] \ll M) \quad \left| \begin{array}{l} 16B = 2^M B \\ \Rightarrow M=4 \end{array} \right.$$

swizzle width = $16 \cdot 2^B$ Bytes

Divide the shared mem region into.

• y : 128-byte bands

• x : 16-byte bands within 128 byte band

• z : byte inside 16-byte chunk.

$$a = 128y + 16x + z;$$

$$0 \leq x < 8 \quad 0 \leq z < 16$$

$$a' = 128y + 16(x \oplus (y \bmod 8)) + z$$

periodic skewing & linear skewing $\rightarrow$ inherent impediment.

no op $\leftarrow$ jump skewing scheme $\rightarrow (A + \lfloor \frac{A}{m} \rfloor) \bmod m$ (use buffer to smooth out access)

skewing storage using XOR $\Rightarrow$ (Scrambled storage).

m-way-interleaved memory system, 2^M banks.

$$m = 2^M$$

- Simple storage $A \bmod m$.
- New scheme $f(A) \bmod m$
 $\hookrightarrow$ Modifies M bits.

Address space

$\mathcal{N} = \{0, 1, \dots, 2^N - 1\}$ set of all N -bit integers

$A_2 \in \mathcal{N}$ $\{A_2 = \text{Binary}\}$

$\Rightarrow A_2 = A$ vector of length N .

for $A_2 \in \mathcal{N}$ $f: \mathcal{N} \rightarrow \mathcal{N}$

$$f(i_0, i_1, \dots, i_n)(A) = A[N-1] A[N-2] \dots A'[i] \dots A[0]$$

where $i \in \{0 \dots M-1\}$

$$\{i_0, i_1, \dots, i_n\} \subseteq \{0 \dots N-1\} \Rightarrow i_0 \neq i_1 \neq \dots \neq i_n$$

$$\hookrightarrow A'[i] = A[i_0] \oplus A[i_1] \dots A[i_n]$$

the function f of g needs to be one-to-one otherwise we again will end up with conflicts.

Theorem $f(i, i_0, i_1, \dots, i_n) : \mathcal{N} \rightarrow \mathcal{N}$ is one-to-one iff $i \in \{i_0, \dots, i_n\}$.

Proof: f only effects the bit i .

Condition 1: Situation 1: Bit other than i is already different,
then ofc $f(A) \neq f(B)$

Situation 2: A & B identical, except bit i .

But as per f

$$f(A)[i] = A_{i_0} \oplus A_{i_1} \dots A_{i_n}$$

$$f(B)[i] = B_{i_0} \oplus B_{i_1} \oplus \dots B_{i_n}$$

$$f(A) \neq f(B) \Leftrightarrow i \text{ is different!}$$

Composition

$$f_i \circ f_j$$

1 $\oplus 4$ (0100) 1 $\oplus 2$ (0010)

$$2^0 = 1$$

$$2^1 = 2$$

$$2^2 = 4$$

$$2^3 = 8$$

$$2^4 = 16$$

$$2^5 = 32$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024$$

$$2^{11} = 2048$$

$$2^{12} = 4096$$

$$2^{13} = 8192$$

$$2^{14} = 16384$$

$$2^{15} = 32768$$

$$2^{16} = 65536$$

$$2^{17} = 131072$$

$$2^{18} = 262144$$

$$2^{19} = 524288$$

$$2^{20} = 1048576$$

$$2^{21} = 2097152$$

$$2^{22} = 4194304$$

... 01. 4 (0100) | 2 (0010)

Element size: $E = 2^e$ bytes

Melms = $4 - \log_2(E) \cdot (M)$

Bytes

uint8_t

uint16

float

16. byte

bit

$\langle 3, 4, 3 \rangle$

$\langle 3, 4, 3 \rangle$

$\langle 3, 3, 3 \rangle$

$\langle 3, 2, 3 \rangle$

$\langle 3, 0, 3 \rangle$

$\langle 3, 7, 3 \rangle$

M determines the indivisible atom.

B determine how many atoms may be permuted.

S determines which higher order bits choose the permutations

Effect (C)

W-TMAP. 128B $\Rightarrow$ $\delta C[ui] \left\lfloor \frac{\text{smem}[ui]}{128} \right\rfloor \cdot 8$

64B $=$ $\delta C[ui] \left\lfloor \frac{\text{smem}[ui]}{128} \right\rfloor \cdot 4$

32B $=$ $\delta C[ui] \left\lfloor \frac{\text{smem}[ui]}{128} \right\rfloor \cdot 2$

128 B $\Rightarrow$

smem[y][x]

$\longleftrightarrow \text{smem}[y][((y+\phi) \times 8)^1 \times x]$

64 B $\Rightarrow$

"

$\longleftrightarrow \text{smem}[y][((y+\phi) \times 4)^1 \times x]$

32 B $\Rightarrow$

"

$\longleftrightarrow \text{smem}[y][((y+\phi) \times 2)^1 \times x]$

0000	0	4 (0100)	2 (0010)
0001	1	0101 (5)	0011 (3)
0010	2	0110 (6)	0000 (0)
0011	3	0111 (7)	0001 (1)
0000	4	0000 (0)	0110 (6)
0101	5	0001 (1)	0101 (7)
0110	6	0010 (2)	0100 (4)
0111	7	0011 (3)	0101 (5)
1000	8	1100 (12)	1010 (10)
1001	9	1101 (13)	1011 (11)
1010	10	1110 (14)	1000 (8)
1011	11	1111 (15)	1001 (9)
1000	12	1000 (8)	1110 (14)
1101	13	1001 (9)	1111 (15)
1110	14	1010 (10)	1100 (12)
1111	15	1011 (11)	1101 (13)

Atom = 2^M chunk,

[Validity condition,
 $B \geq 0, M \geq 0, |S| \geq B$]

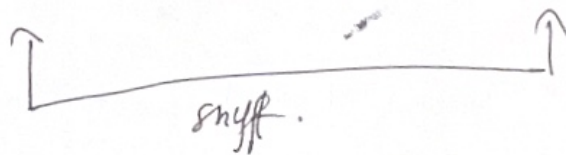
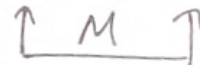
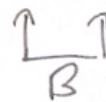
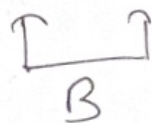
Example: Swizzle $\langle 4, 2, 5 \rangle$

$$a' = a \oplus ((a \gg 7) \ll 15) \ll 2$$

$$addr = [offset \mid Y_{10:7} \mid 6:4 \mid Z_{5:2} \mid X_{1:0}]$$

$$Z'_{5:2} = Z_{5:2} \oplus Y_{10:7}$$

0 6 x x x x x x x x x x x x x x Y Y Y x x x x x x x x Z Z Z x x x x



shared memory has 32 banks, organized such that successive 32-bit words map to successive banks. Each bank has bandwidth of 32 bits per cycle.

